

COM2US INTERNSHIP
PORTFOLIO

감염: 죽음의 도시

트럭을 중심으로 파밍, 적재, 탑승, 무기, 스테이지
전환을 연결한 Unreal Engine 5 C++ 졸업작품
포트폴리오입니다.

박신우

Game Client / Gameplay System Programmer



저는 트럭을 하나의 이동 수단이 아니라, 게임 루프를 묶는 중심 객체로 설계했습니다

플레이어의 행동이 파밍, 적재, 탑승, 무기 사용, 스테이지 전환으로 자연스럽게 이어지도록 트럭을 상태 흐름의 중심에 두고 구조를 재설계했습니다.

01

파밍

공통 상호작용과 아이템 부모 클래스



02

트럭 적재

인벤토리 타입을 트럭 상태로 전달



03

탑승/이동

운전석, 적재칸, 거치총 상호작용 분리



04

2스테이지

타일 기반으로 길을 이어 붙이는 구조

보여줄 역량

시스템을 따로 구현하는 것이 아니라, 상태와 책임을 나누어 플레이 루프가 끊기지 않게 연결했습니다.

설계 근거

상호작용 흐름도, 충돌 원인 분석, 타일 연결 스케치처럼 구현 전에 구조를 먼저 검토한 흔적을 시각 자료로 사용했습니다.

의도적으로 줄인 내용

좀비 AI 세부 구현은 핵심 어필에서 제외하고, 내가 구조를 잡은 트럭/스테이지/상호작용에 집중했습니다.

기능 나열보다 구조 변화와 문제 해결 흐름을 먼저 보여줍니다

결과 화면만 나열하지 않고, 상호작용 상태가 어떻게 확장되고 어떤 병목이 다음 설계 변경으로 이어졌는지를 중심으로 구성했습니다.

기반

상호작용 시스템

IInteractInterface, UInteractTriggerComponent,
ALootItemBase로 파밍 구조 정리

확장

트럭 구조화

운전석/적재칸/거치총 트리거 분리, 캐릭터
부착, 충돌 박스 분리

조작감

거치형 기관총

Yaw/Pitch 축, 카메라 기준 조준, 총구 보정,
탄피/반동 피드백

스테이지

Stage2 타일 구조

Entry/Exit/Trigger 기반의 TileMarker와
TileManager 연결

핵심 구현 파일: ATruck, AMountedMachineGun, AStage2TileManager, UInteractTriggerComponent

파밍은 단순 획득이 아니라, 이후 트럭 적재로 이어질 상태를 만드는 구조입니다

처음에는 인벤토리를 무게값으로만 볼 수 있었지만, 트럭이 어떤 아이템을 실었는지 알아야 했기 때문에 `ItemType` 배열 구조로 바꾸었습니다.

설계 판단

- **공통 인터페이스**
아이템, 트럭, 문처럼 상호작용 가능한 액터를 같은 방식으로 호출
- **범위 기반 등록**
트리거에 들어오면 상호작용 대상을 캐릭터에 등록하고, 벗어나면 해제
- **타입 보존**
아이템을 트럭으로 넘길 때 무게만이 아니라 아이템 종류를 유지

아이템 상호작용 흐름 설계

플레이어와 아이템의 상호작용



트럭 적재 상호작용 흐름 설계

플레이어와 트럭의 상호작용



트럭은 세 가지 역할을 가진 허브로 분리했습니다

같은 F키 상호작용이어도 접근한 위치에 따라 운전, 적재칸
탑승, 거치총 사용이 달라져야 했습니다. 그래서 트럭 내부
상태를 위치/역할 단위로 나눴습니다.

DriverSeat

트럭 조작 권한과 운전석
부착

CargoSeat

적재칸 탑승과 이동 가능
영역

TurretSeat

거치형 기관총 사용 상태

객체 지향 설계

ATruck 생성자에서 Driver/Cargo/Turret 트리거를 분리하고 'ETruckInteractType'으로 분기합니다.
DriverCharacter, MountedWeaponUser 같은 런타임 포인터는 private로 감추고 Getter/Setter로
접근 통로를 제한했습니다.



상호작용 타입 enum

```
UENUM(BlueprintType)
enum class ETruckInteractType : uint8
{
    None,
    LoadCargo, // 1스태이지 적재
    DriverSeat, // 운전석 탑승
    CargoSeat, // 적재함 탑승
};
```

적재칸 탑승 상태 검증



적재칸을 자유 이동 공간으로 만들자, 캐릭터 캡슐이 트럭을 밀어내는 문제가 생겼습니다

캐릭터 캡슐이 차량 PhysicsAsset을 지속적으로 밀어 물리 연산이 불안정해지는 문제였습니다. Collision Profile을 분리해 이동 공간은 막되 차량 본체로 물리 힘이 전달되지 않도록 정리했습니다.

Collision Profile 분리 코드

```
CargoFloorCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoFloorCollision"));
CargoFloorCollision->SetupAttachment(RootComponent);

CargoLeftWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoLeftWallCollision"));
CargoLeftWallCollision->SetupAttachment(RootComponent);

CargoRightWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoRightWallCollision"));
CargoRightWallCollision->SetupAttachment(RootComponent);

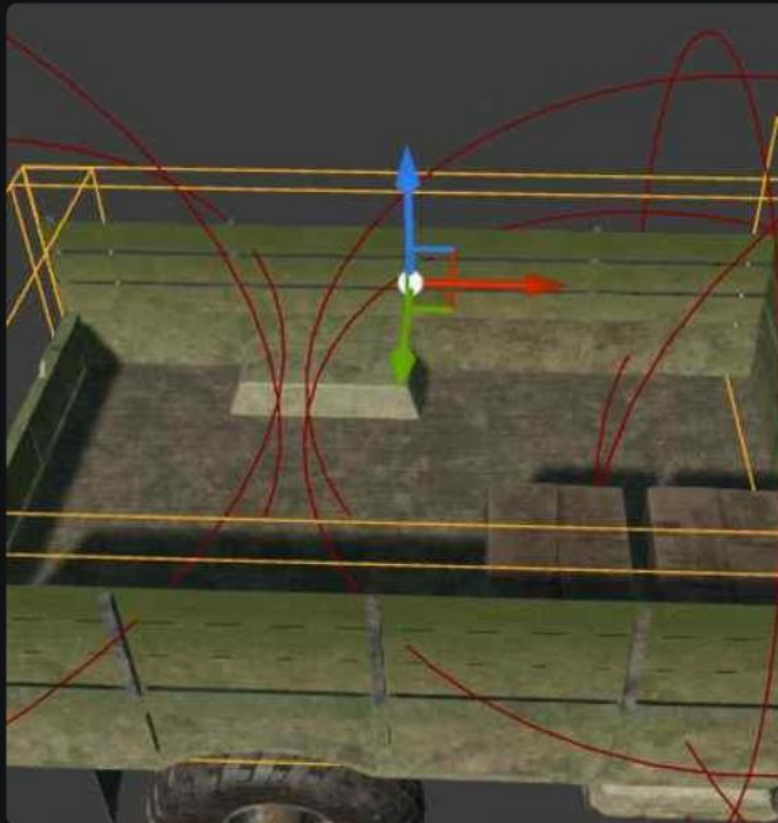
CargoFrontWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoFrontWallCollision"));
CargoFrontWallCollision->SetupAttachment(RootComponent);

CargoBackWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoBackWallCollision"));
CargoBackWallCollision->SetupAttachment(RootComponent);

auto SetupCargoCollision = [](UBoxComponent* Box)
{
    Box->SetCollisionEnabled(ECollisionEnabled::QueryAndPhysics);
    Box->SetCollisionResponseToAllChannels(ECR_Ignore);
    Box->SetCollisionResponseToChannel(ECC_Pawn, ECR_Block);
    Box->SetGenerateOverlapEvents(false);
};

SetupCargoCollision(CargoFloorCollision);
SetupCargoCollision(CargoLeftWallCollision);
SetupCargoCollision(CargoRightWallCollision);
SetupCargoCollision(CargoFrontWallCollision);
SetupCargoCollision(CargoBackWallCollision);
```

적재칸 이동 범위 검증



원인

캡슐이 차량 물리 바디를 계속 밀어 물리 연산이 불안정해짐

해결

적재칸 바닥/벽을 별도 BoxComponent로 만들고 Pawn만 막도록 분리

현재 코드

`QueryOnly` + `ECC\_Pawn Block`으로 이동 제약은 유지하고, 트럭 본체에 불필요한 Physics Impulse가 전달되지 않게 정리

아이템 적재는 런타임 Spawn 대신 미리 배치한 슬롯을 갱신합니다

적재 시점마다 Actor나 Mesh를 새로 생성하면 위치 보정, 수명 관리, 프레임 안정성이 모두 흔들릴 수 있습니다. 그래서 종류별 슬롯을 미리 배치하고 상태 변화 때 표시 여부만 갱신하는 방식으로 바꾸었습니다.

처음

동적 Spawn

적재 때마다 메시를 생성



변경

Pre-placed Slot

Ammo/Fuel/MedKit 슬롯을
미리 배치



결과

Frame 안정화

상태 갱신만 수행해 시각
표현 동기화

최적화 효과

런타임 Actor Spawn 비용을 제거해 순간 Frame Drop 위험을 낮추고, 반복 생성/파괴로 생기는 메모리 오버헤드와 GC 부담을 줄였습니다.

코드 근거

ATruck 생성자에서 AmmoSlot, FuelSlot, MedKitSlot, MountedAmmoSlot 컴포넌트를 만들고 BeginPlay에서 숨김 처리한 뒤 적재 상태에 따라 노출합니다.



동적 생성 대신 에디터 배치 슬롯을 덮어쓰는 방식으로 적재 피드백을 안정화

거치형 기관총은 카메라, 축, 총구를 분리해서 조준감을 만들었습니다

기관총을 단순히 트랙 위에 붙이는 데서 끝내지 않고, 사용자가 보는 방향과 총구 발사 방향이 자연스럽게 이어지도록 컴포넌트 구조를 나눴습니다.

구조

- **YawPivot**
좌우 회전만 담당
- **PitchPivot**
상하 회전과 카메라 붐 기준
- **MuzzlePoint**
블루프린트에서 총구 위치 조정
- **CameraPoint**
기관총 조준 시점 제공

거치형 기관총 컴포넌트 구조

```
const FVector TraceStart = CameraLocation;  
const FVector TraceEnd = TraceStart + CameraRotation.Vector() * 10000.0f;  
  
FHitResult HitResult;  
FCollisionQueryParams QueryParams;  
QueryParams.AddIgnoredActor(this);  
QueryParams.AddIgnoredActor(CurrentUser);  
  
// 라인트레이스 하여 맞으면 hit  
const bool bHit = GetWorld()->LineTraceSingleByChannel(HitResult, TraceStart, TraceEnd, ECC_Visibility, QueryParams);  
// 아무것도 안맞았다면, 멀리 쏘버리기  
const FVector TargetLocation = bHit ? HitResult.Location : TraceEnd;
```



총알이 트럭을 맞거나 조준선과 어긋나는 문제를 단계적으로 고쳤습니다

카메라가 보는 방향, 총구의 실제 위치, 트럭과 사용자의 충돌 판정이 서로 어긋나면 총알이 트럭을 맞거나 조준선 밖으로 튕는 문제가 발생합니다. 화면 기준 조준과 월드 좌표 발사를 분리해 보정했습니다.

1. 카메라 기준 조준

카메라 위치에서 LineTrace로 목표 지점 계산

2. World Transform 보정

MuzzlePoint의 World Transform에서 목표 지점까지 다시 검사해 막힌 지점 보정

3. 충돌 제외

기관총, 사용자, 현재 트럭을 Trace/Projectile에서 무시

4. 발사체 재사용

ObjectPool이 있으면 풀에서 꺼내고 없으면 SpawnActor fallback

Camera Trace + Muzzle 보정 코드

```
if (UObjectPoolSubsystem* PoolSubsystem = GetWorld()->GetSubsystem<UObjectPoolSubsystem>())
{
    if (AActor* PooledActor = PoolSubsystem->SpawnFromPool(ProjectileClass, FireLocation, FireRotation))
    {
        if (AFPSProjectile* Projectile = Cast<AFPSProjectile>(PooledActor))
        {
            Projectile->SetOwner(this);
            Projectile->SetInstigator(CurrentUser);
            Projectile->FireInDirection(FireDirection);
        }
    }
}
else
{
    if (AFPSProjectile* Projectile = GetWorld()->SpawnActor<AFPSProjectile>(ProjectileClass, FireLocation, FireRotation))
    {
        Projectile->SetOwner(this);
        Projectile->SetInstigator(CurrentUser);
        Projectile->FireInDirection(FireDirection);
    }
}
```

트럭에 타는 순간, 조작 대상과 캐릭터 표현이 동시에 바뀌어야 했습니다

트럭 조작은 가능하지만 캐릭터 표현이 운전석으로 따라오지 않는 상태 전환 문제가 있었습니다. 원인은 탑승 순간 Possess 대상이 바뀌며 'IsLocallyControlled' 흐름이 달라진 것이었습니다.

해결 방식

운전석 탑승 호출 시 캐릭터를 DriverSeatPoint에 부착하고, 트럭에 부착된 캐릭터는 다른 플레이어 처리 코드가 잘못 개입하지 않도록 분기했습니다.

카메라 좌표 처리

우클릭 줌과 크로스헤어 발사 보정도 같은 맥락에서, 화면 중앙을 'DeprojectScreenPositionToWorld'로 변환해 UI 조준점과 실제 발사 방향을 맞췄습니다.



상태 전환 분석

문제 원인을 '컨트롤 대상 변경'까지 추적

면접 포인트

Possess, Attach, Animation, Local Control 흐름 설명 가능

Stage2는 손으로 그린 Entry / Exit / Trigger 구조에서 출발했습니다

트럭이 다음 타일 트리거를 밟으면, 현재 타일의 Exit 위치에 다음 타일의 Entry가 맞물리도록 설계했습니다. 손으로 정리한 연결 규칙을 TileMarker와 TileManager 책임으로 분리했습니다.

TileMarker

Entry Arrow

타일의 시작 기준점



Trigger

Next Tile Trigger

트럭 진입 시 이벤트 발생



TileManager

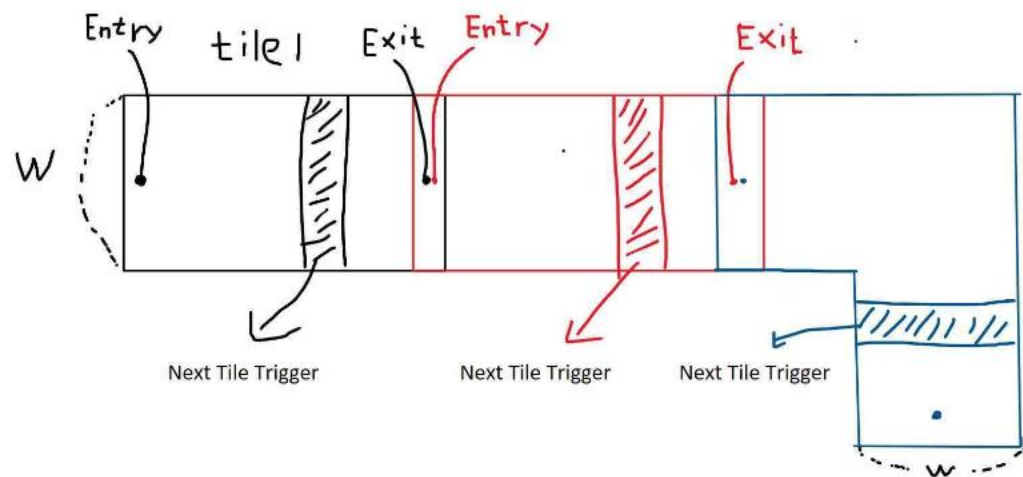
Exit → Entry

다음 타일 위치 계산

구현 책임

TileMarker는 타일 내부 기준점을 제공하고, TileManager는 어떤 타일을 언제 활성화할지 결정하도록 책임을 나눴습니다.

Entry / Exit / Trigger 연결 설계



비동기 로드의 빈틈을 ActiveTiles와 초기화 단계로 관리했습니다

비동기 레벨 스트리밍에서는 로드 요청 직후 타일 내부 Actor나 Marker를 바로 참조할 수 없습니다. 그래서 로드 요청과 Marker 확보를 분리하고, 초기화 확정 단계를 별도로 두었습니다.

TileMarker

구성

```
SceneRoot = CreateDefaultSubobject<USceneComponent>(TEXT("SceneRoot"));
SetRootComponent(SceneRoot);

EntryArrow = CreateDefaultSubobject<UArrowComponent>(TEXT("EntryArrow"));
EntryArrow->SetupAttachment(SceneRoot);
EntryArrow->ArrowColor = FColor::Green;

ExitArrow = CreateDefaultSubobject<UArrowComponent>(TEXT("ExitArrow"));
ExitArrow->SetupAttachment(SceneRoot);
ExitArrow->SetRelativeLocation(FVector(5000.0f, 0.0f, 0.0f));
ExitArrow->ArrowColor = FColor::Yellow;

NextTileTrigger = CreateDefaultSubobject<UBoxComponent>(TEXT("NextTileTrigger"));
```

ActiveTiles 메타데이터

```
// Load된 Tile의 정보를 저장한다. 나중에 모든 완료된 Tile을 Finalize할 때 사용

// ActiveTiles 배열에 새 원소를 추가하고, 그 참조를 가져온다
FStage2LoadedTile& LoadedTile = ActiveTiles.AddDefaulted_GetRef();
LoadedTile.SourceLevel = TileLevel; // 어떤 레벨 에셋에서 로드된 타일인지 저장
LoadedTile.StreamingLevel = StreamingLevel;
LoadedTile.TileType = TileType;
LoadedTile.RequestedEntryTransform = SpawnTransform;
LoadedTile.AppliedLevelTransform = LevelTransformToApply;
LoadedTile.bInitialized = false;

if (bVerboseLog)
{
    UE_LOG(LogTemp, Log, TEXT("Stage2TileManager: Requested load for %s | Requested=%s | Applied=%s"),
        *TileLevel.ToSoftObjectPath().ToString(),
        *SpawnTransform.GetLocation().ToString(),
        *LevelTransformToApply.GetLocation().ToString());
}
```

1. 요청

ULevelStreamingDynamic으로 타일 레벨 로드 요청

2. 보류

ActiveTiles에 SourceLevel, StreamingLevel, Transform 저장

3. 확정

Tick에서 TryFinalizeLoadedTiles로 Marker 확보

4. 연결

Trigger 이벤트가 들어오면 다음 SpawnTransform 계산

Frame Drop를 줄이기 위해 타일 로드와 재사용 시점을 분리했습니다

트리거를 밟는 순간 다음 맵을 로드하면 스트리밍 비용이 플레이 중간에 발생합니다. Stage2 진입 시 타일 풀을 선로드하고, 지나간 타일은 삭제 대신 재배포해 런타임 비용을 분산했습니다.

성능 저하 인식

Trigger-time Load → 순간 Frame Drop 가능성

선로드

Stage2 진입 시 'PreloadedTilesPerType' 기준으로 TilePool 구성

재사용

지나간 타일 삭제 대신 PoolParkingTransform으로 되돌려 재배포

Low-level 관점

FStage2LoadedTile처럼 포인터, FTransform, bool 플래그가 섞인 구조는 sizeof, 필드 순서, bit packing을 기준으로 패딩/캐시 효율을 검토



제가 보여주고 싶은 강점은 기능 구현보다 구조를 끝까지 연결하는 태도입니다

이 프로젝트에서 저는 파밍, 트럭, 적재, 거치총, 스테이지 전환이 하나의 플레이 경험으로 이어지도록 시스템의 책임을 나누고, 문제가 생길 때마다 구조를 다시 잡았습니다.

역할

Gameplay 구조 설계

강점

문제 원인 추적과 재설계

태도

문제 원인과 설계 선택을
코드로 설명 가능

면접에서 설명 가능한 코드

ATruck: 상호작용 트리거, 적재 슬롯, 탑승/하차, 충돌 분리

AMountedMachineGun: 조준 보정, 발사체 풀링, 반동/탄피 피드백

AStage2TileManager: TileMarker 이벤트, ActiveTiles, TilePool, 비동기 로드 확정

